
Table of Contents

Teoria	1
Llibre	1
Practica 6	3
Practica 5	3
Practica 4	5
Practica 3	6

Teoria

```
%{  
Formula de la recta:  
y=a+mn  
- a: intersecció amb eix y  
- m: pendent  
  
y-y_o = m*(x-x_o)  
- y_o, x_o: punt per on passa la recta  
- m: pendent  
%}
```

Llibre

```
% Code 1: Bisection method for solving  $f(x) = 0$  in  $[a,b]$   
% Input: 1.  $[a,b]$ : interval (it assumes that  $f(a)f(b) < 0$ )  
% 2. tol: tolerance so that  $\text{abs}(x_{k+1} - x_k) < \text{tol}$   
% 3. itmax: maximum number of iterations allowed  
% 4. fun: function's name  
% Output: 1. xk: resulting sequence  
% 2. res: resulting residuals  
% 3. it: number of required iterations  
function [xk,res,it] = bisection(a,b,tol,itmax,fun)  
ak = a; bk = b; xk = []; res = [];  
it = 0; tolk = abs(bk-ak)/2 ;  
while it < itmax & tolk > tol  
ck = (ak + bk)/2; xk = [xk ck];  
if it > 0; tolk = abs(xk(end)-xk(end-1)); end  
fa = feval(fun,ak); fc = feval(fun,ck); res = [res abs(fc)];  
if fc*fa < 0; bk = ck; else ak = ck; end  
it = it + 1 ;  
end  
end  
  
% Code 2: Newton's method for solving  $f(x) = 0$   
% Input: 1. a: initial guess  
% 2. tol: tolerance so that  $\text{abs}(x_{k+1} - x_k) < \text{tol}$   
% 3. itmax: maximum number of iterations allowed  
% 4. fun: function's name
```

```

% 5. dfun: derivative's name
% (If dfun = '0', then f'(x) is approximated)
% Output: 1. xk: resulting sequence
% 2. res: resulting residuals
% 3. it: number of required iterations
function [xk,res,it] = newton(a,tol,itmax,fun,dfun)
xk = a; fk = feval(fun,xk); res = abs(fk); it = 0;
tolk = res(1); dx = 1e-8 ;
while it < itmax && tolk > tol
if dfun == '0'
dfk = (feval(fun,xk(end)+dx)-fk)/dx;
else
dfk = feval(dfun,xk(end));
end
xk = [xk, xk(end) - fk/dfk]; fk = feval(fun,xk(end));
res = [res abs(fk)]; tolk = abs(xk(end)-xk(end-1));
it = it + 1;
end
end

% Code 3: Lagrange cardinal polynomials (equispaced nodes)
function [P,xn,z] = cardpolequi(a,b,n,m)
% Input: 1. a & b: interval [a,b]
% 2. n: with x_0 = a and x_n = b (n+1 interp nodes)
% 3. m: number of points in dense grid
%
% Output: 1. P: matrix of card. polynomials.
% 2. xn: equispaced interpolation nodes
% 3. z: dense grid
xn = a + [0:n]'.*(b-a)/n; z = linspace(a,b,m)';
% Matrix containing cardinal polynomials
P = ones(m,n+1) ;
for jj = 0:n
knj = [0:jj-1 jj+1:n];
for kk = knj
P(:,jj+1) = P(:,jj+1).*(z-xn(kk+1))/(xn(jj+1)-xn(kk+1));
end
end
end
% If fn is a column vector containing f(x) at the nodes xn,
% then P*fn provides the interpolant P_n f(z) on the dense
% grid z (see Practical 3 for details)

% Code 4: Chebyshev Barycentric Interpolation
% Chebyshev nodes of second kind (practical abscissas)
function ff = barifun(xn,fn,z)
% Input: 1. xn = interpolation Chebyshev nodes
% mapped to [a,b] (column vector):
% x_j = a + (b-a)*(1+cos(j*pi/n))/2; j=0,1,...,n
% 2. fn = function at nodes xn (column vector)
% 3. z = evaluation grid (z /= xn; column vector)
% Output: ff interpolated function at z
n = length(xn); ff = 0*z; num = ff ; den = ff ;

```

```

s = (-1).^[0:n-1]';
sn = s.*[fn(1)/2; fn(2:end-1); fn(end)/2];
sd = [s(1)/2; s(2:end-1); s(end)/2];
for ii = 1:length(num);
num = ((z(ii) - xn).^(-1))'*sn;
den = ((z(ii) - xn).^(-1))'*sd;
ff(ii) = num/den;
end
end

```

Not enough input arguments.

Error in funciones_parcial (line 25)
 ak = a; bk = b; xk = []; res = [];

Practica 6

```

% Interpolación baricéntrica
% n: numallaero de nodos
% node: conjunto de nodos en una cierta distribución
% malla: 1 punto de la mallaalla densa
function g = interpol_bar(n, node, m)
    len_m = size(m, 2);

    s = (-1).^(0:n);
    s_n = s.*[f(node(1))/2, f(node(2:end-1)), f(node(end))/2];
    s_d = [s(1)/2, s(2:end-1), s(end)/2];
    g = [];
    for k= 1: len_m
        num = s_n*((m(k)-node).^(-1))';
        den = s_d*((m(k)-node).^(-1))';
        g = [g, num/den];
    end
end

% Nodos de Chebychev
function c = cheby(n)
    c = [];
    for j = 0:n
        c = [c, cos((j*pi)/n)];
    end
end

% Funció random
function f = f(x)
    f = tanh(20*sin(12*x))+(2/100)*exp(3*x).*sin(300*x);
end

```

Practica 5

```

% Matriu de lagrange
% Donats n nodes i una malla densa de m punts, retorna la matriu de
% lagrange

```

```

function M = mat_lag(n, m)
    x = generar_punts(n);
    z = generar_punts(m);
    M = zeros(m+1, n+1);
    for j = 1:m+1
        for k = 1:n+1
            M(j, k) = lag_card(x, z(j), k, n);
        end
    end
end

% Donats n nodes i malla fina de m nodes, ploteja els polinomis de lagrange
function plot_lag(n, m)
    x = generar_punts(n);
    z = generar_punts(m);
    M = mat_lag(n, m);
    hold on;
    for j = 1:n+1
        plot(z, M(:, j));
    end
    hold off;
end

% Genera n punts equiespaiats
function pts = generar_punts(n)
    pts = -1;
    for i = 1:n
        x_i = -1 + (2*i/n);
        pts = [pts, x_i];
    end
end

% Donada la matriu de lagrange, retorna un vector corresponent a la funció
% de Lebesgue
function leb = lebesgue(M)
    [m, n] = size(M);

    mat = ones(n);
    vect = mat(:, n);

    leb = abs(M)*vect;
    leb = leb';
end

% Donats n nodes, un punt de la malla densa, un index i i el nombre de nodes
% n,
% retorna el cardinal de lagrange del punt de la malla densa i l'índex i
function lag_card = lag_card(x, z, i, n)
    lag_card = 1;
    for j = 1: n+1
        if j ~= i
            lag_card = lag_card * ((z-x(j))/(x(i)-x(j)));
        end
    end
end

```

```
end
end
```

Practica 4

```
% Donada una funció, un punt i un paràmetre del qual depèn la funció,
% aproxima la derivada de la funció en el punt x0
function deriv = deriv_param(f, x0, param)
    Ax = 10 ^ -10; % arbitrary small number, to represent infinitesimal
    increase
    deriv = (f(x0 + Ax, param) - f(x0, param)) / Ax;
end

% Donat un punt inicial, una tolerància objectiu, un nombre d'iteracions
% màxim, una funció i un paràmetre del qual depèn la funció, retorna els
% vectors amb els punts estudiats, el valor de la funció en el punt estudiat
% i el nombre d'iteracions.
function [xk, fk, it] = newtonParam(x1, tol, itmax, fun, param)
    %x1: punt inicial      tol: interval tolerancia
    it = 0;
    xk = x1;
    fk = [fun(x1, param)];
    while (abs(fun(xk(end), param)) > tol) && (it < itmax)
        new_x = xk(end) - (fun(xk(end), param) / deriv_param(fun, xk(end),
param));
        xk = [xk, new_x];
        fk = [fk, fun(xk(end), param)];
        it = it + 1;
    end
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%
% Ajusta una recta  $y = a x + b$  als vectors  $x, y$ .
%
% Torna el coeficient de regressió  $r$ , el pendent  $a$  i
% l'ordenada a l'origen  $b$ .
%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

function [r,a,b]=reg_lin(x,y)

if iscolumn(x)
    x=x';
end

if iscolumn(y)
    y=y';
end

if size(x,2) ~= size(y,2)
    fprintf('La mida dels dos vectors no es la mateixa al cridar a reg_lin. \n')
    return
end
```

```

end

n=size(x,2);

ax=sum(x)/n; ay=sum(y)/n;

ax2=sum(x.^2)/n; ay2=sum(y.^2)/n; axy=sum(x.*y)/n;

a=(axy-ax*ay)/(ax2-ax^2);
c=(axy-ax*ay)/(ay2-ay^2);
r=sqrt(a*c);
b=ay-a*ax;
d=ax-c*ay;

%n,r,a,b,ax,ay,ax2,ay2,axy;

end

```

Practica 3

```

% Donats un punt inicial, una tolerància màxima, un nombre màxim
% d'iteracions i la funció a estudiar, retorna els punts estudiats, el
% valor de la funció en aquests punts i el nombre d'iteracions fetes.
% Aquesta versió no depèn de paràmetres
function [xk, fk, it] = newtonA(x1, tol, itmax, fun)
    %x1: punt inicial      tol: interval tolerancia
    it = 0;
    xk = [x1];
    fk = [fun(x1)];
    while (abs(fun(x1)) > tol) && (it < itmax)
        new_x = x1 - (fun(x1) / deriv(fun, x1));
        x1 = new_x;
        xk = [xk, x1];
        fk = [fk, fun(x1)];
        it = it + 1;
    end
end

% Donats una funció i un punt inicial, retorna una aproximació de la
% derivada de la funció el punt x0.
function deriv = deriv(f, x0)
    Ax = 10 ^ -10; % arbitrary small number, to represent infinitesimal
    increase
    deriv = (f(x0 + Ax) - f(x0)) / Ax;
end

```

Published with MATLAB® R2024a